RICE UNIVERSITY

# GPU Architecture

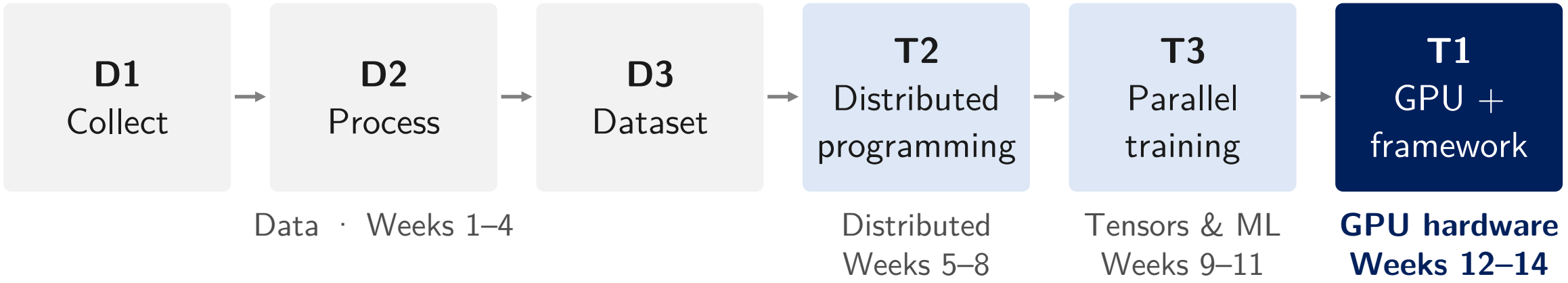
Why GPUs are fast, where they are slow, and how to predict which one you will get

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 12, Lecture 1

# Where we are in the pipeline

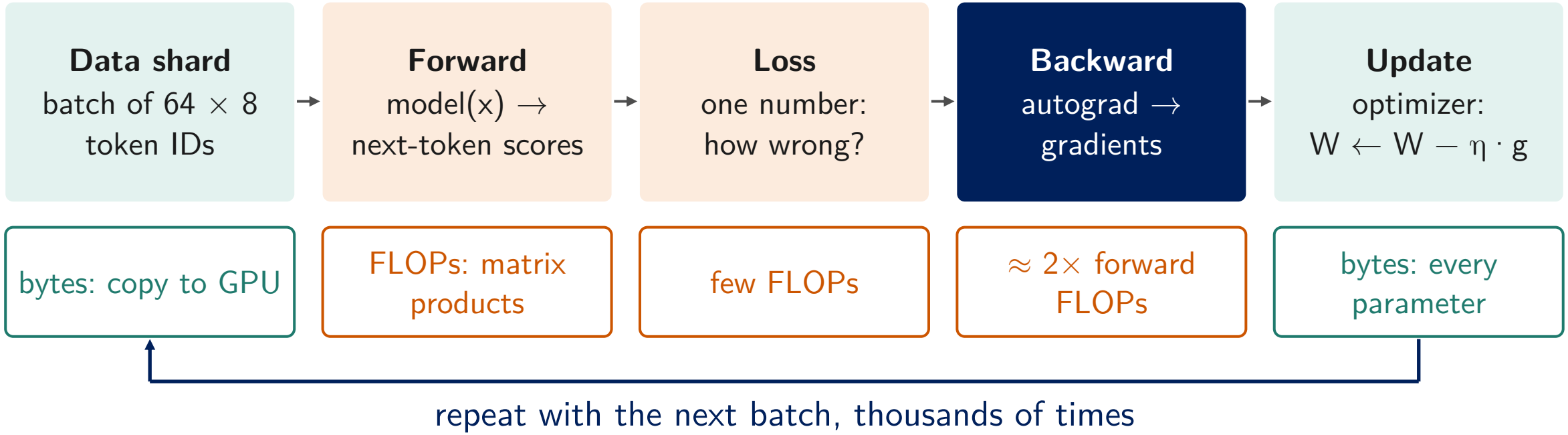



**In our pipeline**

Today: what happens inside **one GPU** running **one kernel** — the foundation for T1 and Lab 3.

Next: one training step again, and where the GPU fits among Weeks 5–14.

# What one training step does




**Key idea**

Training time = number of steps × time per step. Weeks 5–11 split it across **many GPUs**; Weeks 12–14 make **each GPU fast**.

# Weeks 5–14 at a glance

| Problem | | System component | Where |
|---|---|---|---|
| One GPU is too slow or too small | → | **Processes + communication: MPI, NCCL** | Wk 5–8 · T2 |
| How do we split the work? | → | **Data, tensor, pipeline parallelism** | Wk 9–11 · T3 |
| One step is millions of multiply-adds | → | **GPUs and kernels** | Wk 12–14 · T1 |

dark: today

# Why do we need GPUs?

**The job:** train GPT-3 (175 billion parameters)
$\approx$ **3.14 $\times$ $10^{23}$** floating-point operations, almost all multiply-adds

**If one CPU core did all of this work, how long would it take?**

One **CPU core**, at its peak
$\approx$ **52,000 years**

One **H100 GPU**, at its peak
$\approx$ **10 years**

**Key idea**

A GPU is **thousands of times** faster than a CPU core at this work — and even one GPU is too slow. So: many GPUs in T2 and T3, and fast GPUs in T1.

Source: Brown et al., NeurIPS 2020 (GPT-3 compute); CPU: assumed 3 GHz core, 2 AVX-512 FMA units; H100 datasheet (SXM5), BF16 peak.

# What a GPU is, in plain words

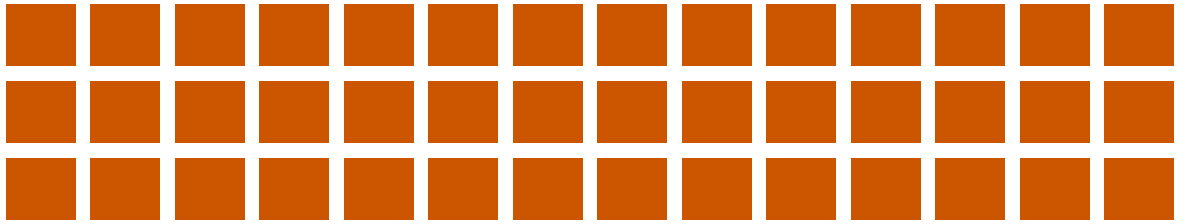
**CPU (host)**
runs your Python
a few big cores

host memory (DRAM)

PCIe link

$\approx$ 64 GB/s

**GPU (device)**: thousands of simple arithmetic lanes



HBM: the GPU's own memory, $\approx$ 3.35 TB/s

1. Copy the data to HBM once, and keep it there.
2. Launch kernels: "run z = x + y on all $10^8$ elements."

**Key idea**

A **kernel**: one function the GPU runs on many elements at once. The CPU **launches** it.

Source: H100 datasheet (SXM5), HBM bandwidth; PCIe 5.0 x16, per direction.

# Where GPUs show up

- **Graphics first:** every pixel is independent, so the chip is many simple cores.
- **2007, CUDA:** those cores become programmable for any arithmetic.
- **2012, AlexNet:** trained on two GPUs. Llama 3 used up to **16,384** H100s.
- **In PyTorch:** `x.to("cuda")`; every op on `x` then runs as GPU kernels.

**The price**

Fast only with **lots of independent work**

Data must fit in **80 GB** of GPU memory

Each launch costs CPU time: tiny jobs are **overhead-bound**

**In our pipeline**

**Why GPUs?** Training is a huge number of independent multiply-adds, and a GPU runs tens of thousands at once. In T1 and Lab 3 your kernels run on one.

Source: Krizhevsky et al., NeurIPS 2012; Llama Team, "The Llama 3 Herd of Models," 2024; H100 datasheet (SXM5).

# Learning goals for today

- Explain why a GPU trades **latency** for **throughput**.
- Map **thread** → **warp** → **block** → **grid** onto the GPU's **SMs**.
- Name the memory levels — **registers, shared memory, L2, HBM** — with rough sizes and speeds.
- Compute a kernel's **arithmetic intensity** from FLOPs and bytes.
- Use the **roofline model** to predict compute- versus memory-bound, and estimate runtime **before** measuring.

# A puzzle to start

**Kernel A — vector add**

$z = x + y$, $n = 10^8$ floats

1 add per element

**Kernel B — matrix multiply**

$C = A \cdot B$, $4096 \times 4096$

8,192 FLOPs per output

**On the same GPU, which kernel reaches a larger fraction of the advertised peak FLOP/s?**

**Key idea**

**Kernel B** gets close to peak; **kernel A** reaches well under 1% of it. For A, arithmetic is not the bottleneck — **moving the data is**.

# Counting work: FLOPs

- **FLOP** = one floating-point add or multiply; **FLOP/s** is a rate.
- Dot product of length K: K multiplies + (K − 1) adds ≈ **2K FLOPs**.
- C = A · B with A (M × K), B (K × N): M · N dot products.

$$F = 2 \cdot M \cdot N \cdot K$$

$$M = N = K = 4096$$

$$F \approx 1.37 \times 10^{11}\ \text{FLOPs}$$

**Compute**

At 989 TFLOP/s this takes ≈ **0.14 ms** — *if* arithmetic were the only cost.

Source: H100 datasheet (SXM5), BF16 tensor-core peak without sparsity.

# Counting work for our own model

| Step (batch B = 64) | FLOPs |
|---|---|
| Embedding lookup | $\approx 0$ (memory only) |
| Hidden: $(64\times128)\cdot(128\times64)$, tanh | $\approx 1.05$ M |
| Logits: $(64\times64)\cdot(64\times322)$ | $\approx 2.64$ M |
| Softmax + cross-entropy | $\approx 0.1$ M |
| Backward $\approx 2\times$ matmuls | $\approx 7.4$ M |
| **One training step** | **$\approx$ 11 MFLOP** |

ModelConfig: vocab 322, context 8, embedding 16, hidden 64

**34,338 parameters** $\approx$ 137 KB (FP32)

**Check**

At 67 TFLOP/s, 11 MFLOP takes $\approx$ **0.17 µs**. Hold that thought.

Part 1

# From CPU to GPU

Why build a processor with thousands of slow, simple cores?

# Latency versus throughput

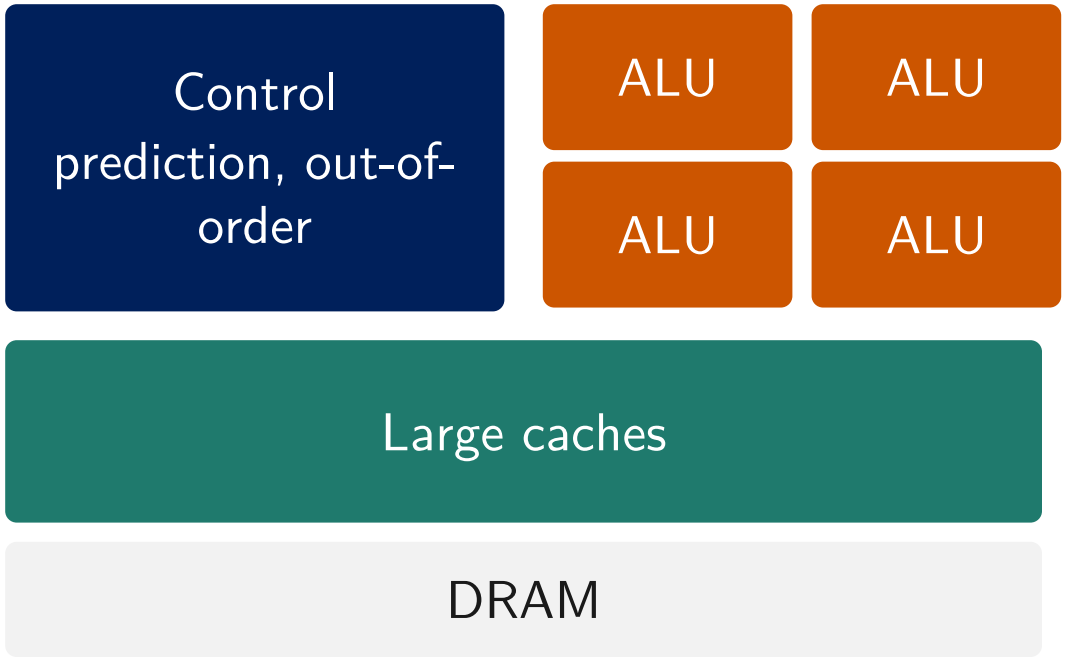
| | **Latency-oriented (CPU)** | **Throughput-oriented (GPU)** |
|---|---|---|
| Goal | Finish **one** task fast | Finish **many** tasks per second |
| Cores | Tens of large cores | Thousands of simple lanes |
| Chip area | Caches, prediction, control | Arithmetic and registers |
| Hides slow memory by | Big caches, speculation | Switching to other threads |

**Key idea**

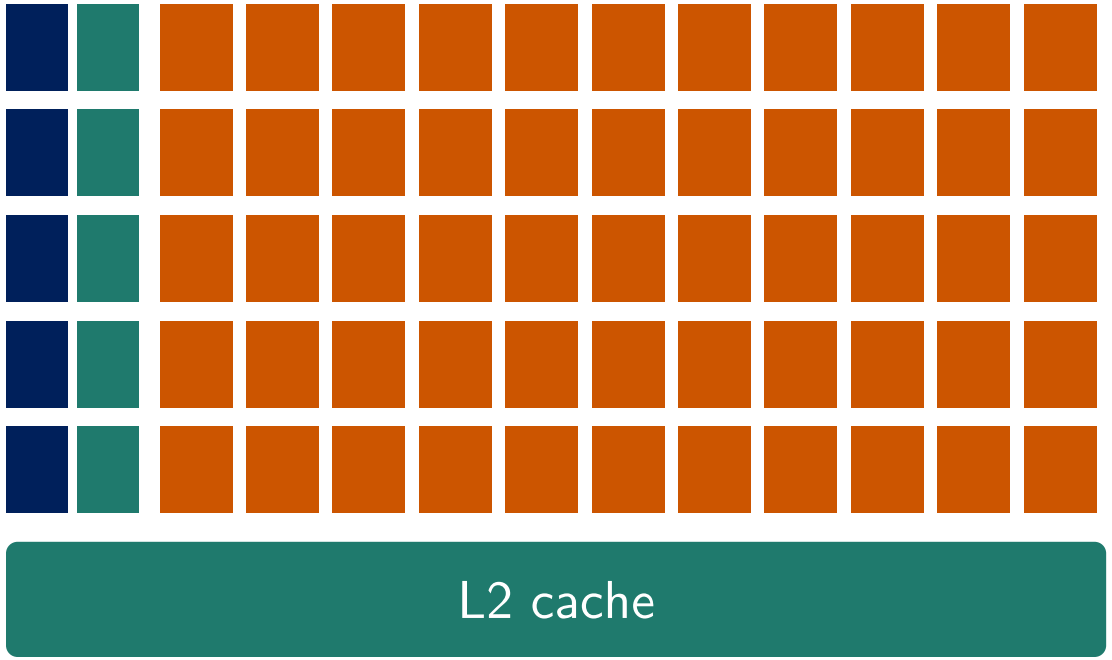
A sports car versus a fleet of buses. Training is a **throughput** problem.

# Where the transistors go

## CPU: a few powerful cores




## GPU: many simple lanes

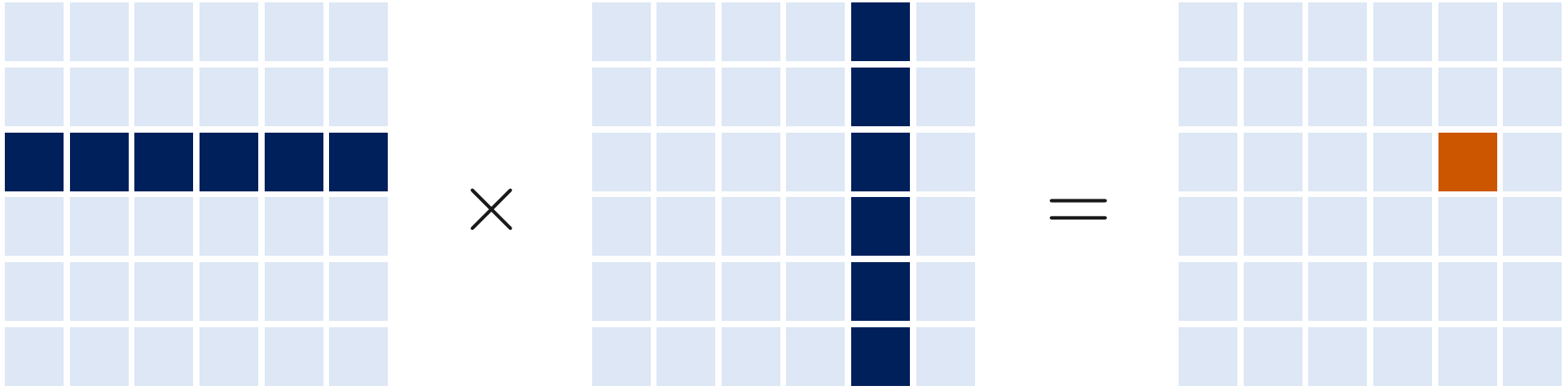



blue: control · green: cache · orange: arithmetic

**Key idea**

The GPU spends its area on arithmetic, so it needs **many threads** in flight to stay busy.

# Why training fits the GPU



**A** row i × **B** column j = **C** C[i, j]

**Key idea**

Every C[i, j] is **independent**: $4096^2 \approx$ **16.8 million** tasks that can run at once.

# Amdahl's law: the serial part limits you

$$\text{new time} = (1 - p) \cdot T + p \cdot T / s$$

$$\text{speedup} = 1 \; / \; ( \; (1 - p) + p \; / \; s \; )$$

$$s \to \infty: \quad \text{speedup} \to 1 \; / \; (1 - p)$$

p = 0.50 $\to$ at most 2×

p = 0.90 $\to$ at most 10×

p = 0.95 $\to$ at most 20×

p = 0.99 $\to$ at most 100×

**Watch out**

If 5% of a training step is Python on the CPU, **no GPU** can make it more than 20× faster.

# Check yourself

Two minutes with a neighbour.

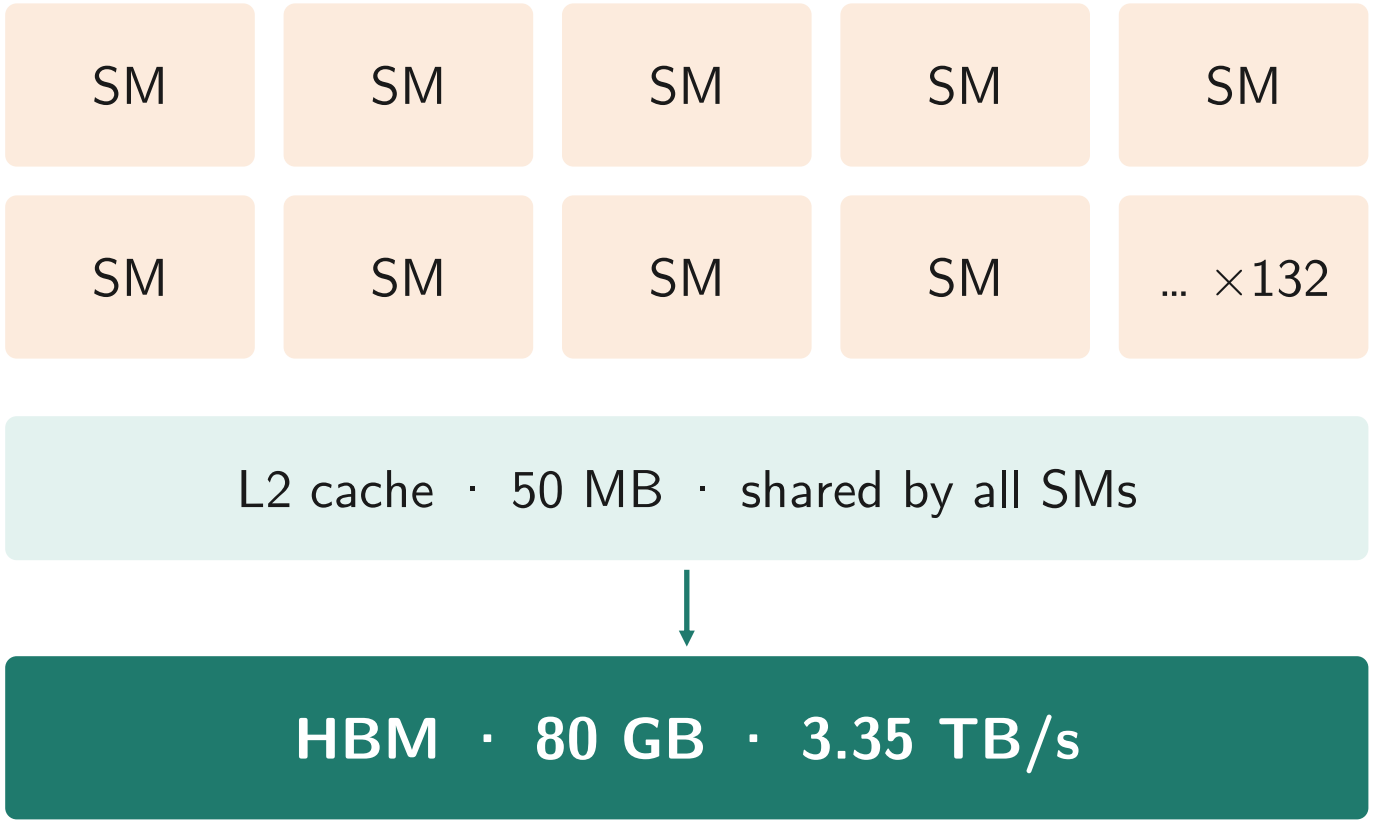
- Is training a latency problem or a throughput problem? Why?
- Name a computation that is hard to parallelize on a GPU, and say why.
- 80% of a program moves to a GPU that runs that part 100× faster. What is the overall speedup?

Part 2

# The SIMT execution model

How does one program become 100,000 threads?

# A GPU is a grid of streaming multiprocessors




$$\begin{aligned}
&\text{Peak FP32} \\
&= \text{SMs} \times \text{lanes} \times 2 \times \text{clock} \\
&= 132 \times 128 \times 2 \times 1.98\ \text{GHz} \\
&\approx 67\ \text{TFLOP/s}
\end{aligned}$$

A fused multiply-add $a \cdot b + c$ counts as 2 FLOPs.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

# Inside one SM

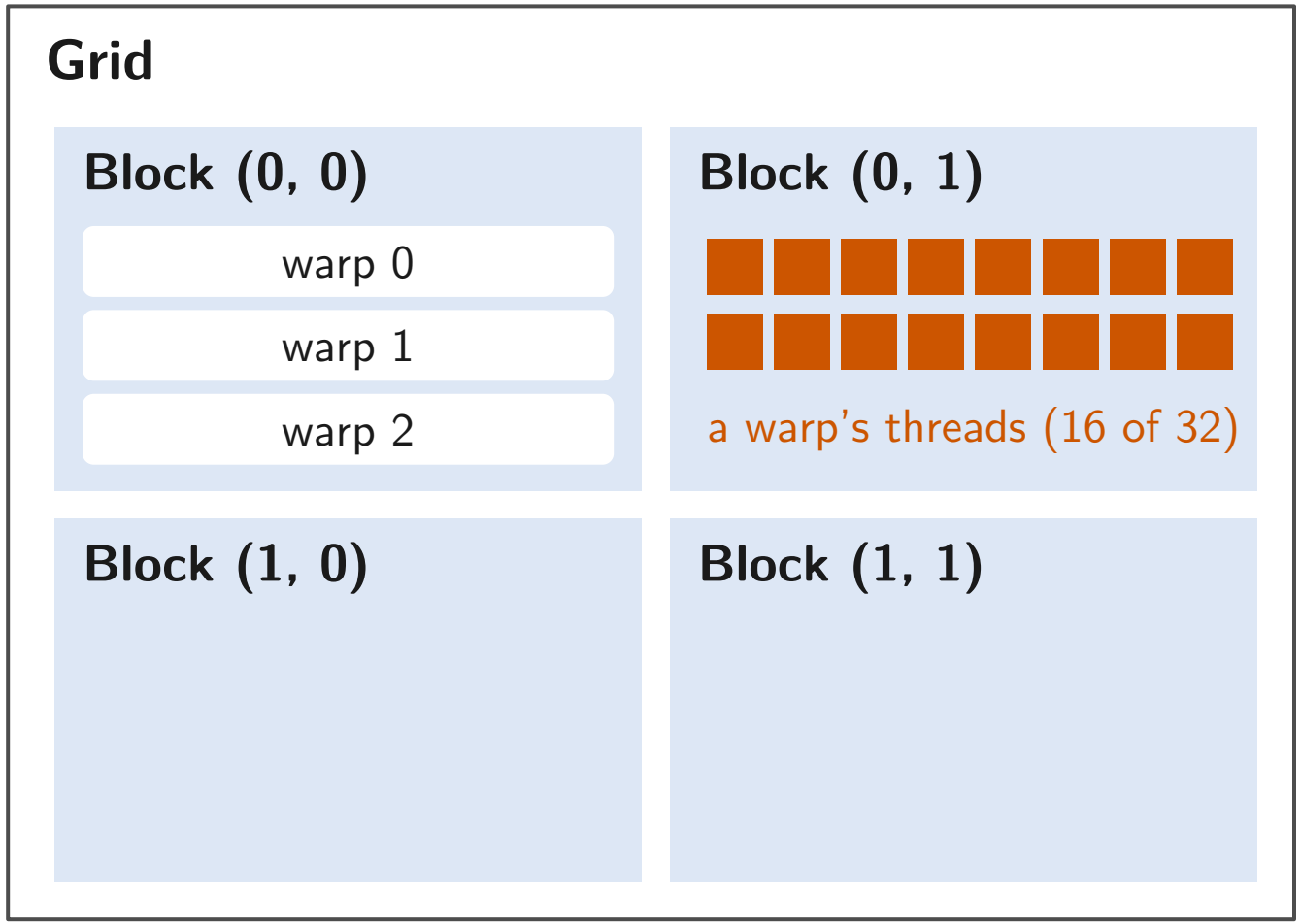
One of 4 sub-partitions

| | |
|---|---|
| Warp scheduler | → Picks a ready warp and issues **one instruction** each cycle. |
| Registers · 64 KB | → Private, fastest storage: each thread has its own. |
| 32 FP32 lanes | → Execute that instruction for **32 threads at once**. |
| **Tensor core** | → Multiplies small matrix tiles in one step (TF32, BF16, FP8, …). |

× 4 sub-partitions share 256 KB of L1 / shared memory (up to 228 KB shared)

Source: NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022), SM description.

# Software view: grid → block → warp → thread




**Grid** — every block of one launch; blocks must be **independent**.

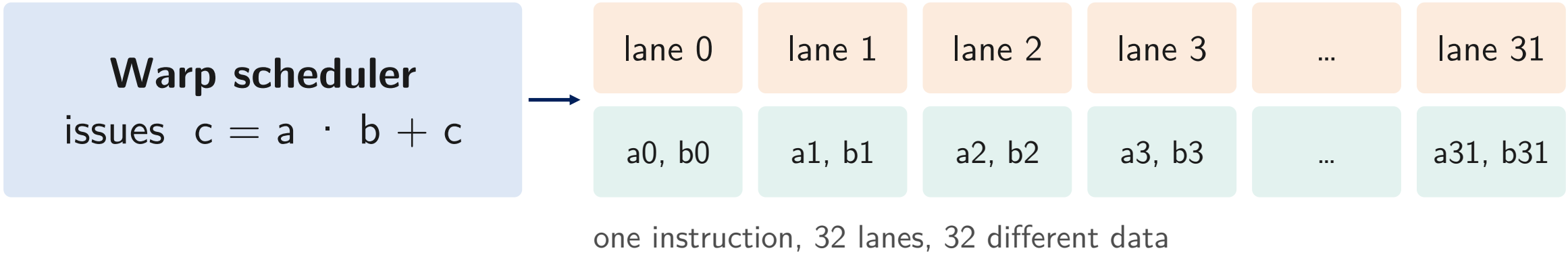
**Block** — up to 1024 threads on **one SM**; shares memory; can synchronize.

**Warp** — 32 threads that the hardware runs **together**.

**Thread** — one instance of the kernel, with its own registers.

Source: NVIDIA CUDA C++ Programming Guide: warp size 32; at most 1024 threads per block.

# SIMT: single instruction, multiple threads




- One fetch and decode drives **32** operations $\rightarrow$ cheap control.
- Each thread has **its own registers** and index; code is written for one thread.
- A warp computes $(f(x_0), f(x_1), \ldots, f(x_{31}))$ with one instruction.

# Branch divergence

```
i = thread_index()
if x[i] > 0:          # lanes 0-15
    y[i] = slow_a(x[i])
else:                 # lanes 16-31
    y[i] = slow_b(x[i])
```

$$\text{efficiency} = \text{active lanes} \;/\; 32$$

$$50/50 \text{ split:}$$

$$\text{time} \approx \mathcal{T}(A) + \mathcal{T}(B)$$

**Pass 1: lanes run branch A**



**Pass 2: lanes run branch B**

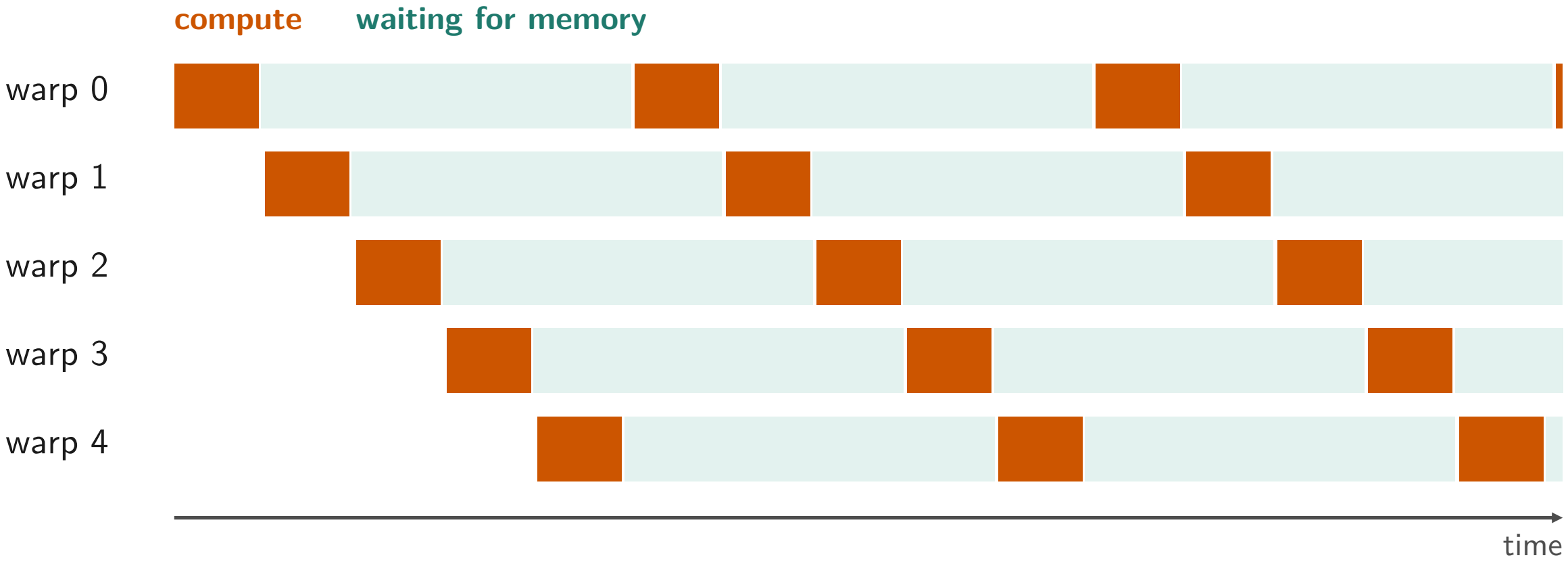


orange: active lane · grey: masked off

**Watch out**

Divergence costs time only **within a warp**; different warps may take different paths freely.

# Hiding latency by switching warps




**Key idea**

While warp 0 waits, warps 1–4 compute; switching is free.

# How many warps are enough? Little's law

$$\text{in flight} = \text{bandwidth} \times \text{latency}$$

$$= 3.35\ \text{TB/s} \times 600\ \text{ns} \approx 2\ \text{MB}$$

$$\text{per SM: } 2\ \text{MB} \,/\, 132 \approx 15\ \text{KB}$$

- 4-byte load per thread $\rightarrow \approx$ **3,800** threads per SM needed.
- An SM holds at most **2,048** threads $\rightarrow$ not enough.
- Fix: **16-byte** loads, several in flight per thread.

**Key idea**

Hiding latency needs **concurrency**: enough bytes in flight to cover the wait.

Source: Little (1961); H100 datasheet (bandwidth); latencies are rounded, from microbenchmarks (e.g. Luo et al., IPDPS 2024).

# Occupancy: a worked example

Kernel: 256 threads per block · 128 registers per thread · 48 KB shared memory per block

- Registers: $65{,}536 \div 128 = 512$ threads = **16 warps**
- Shared memory: $\lfloor 228\ \text{KB} \div 48\ \text{KB} \rfloor = 4$ blocks $\times$ 8 warps = **32 warps**
- Thread limit: $2{,}048 \div 32 =$ **64 warps**
- Binding constraint: $\min(16, 32, 64) = 16$ warps $\rightarrow$ **25% occupancy**

**Watch out**

Occupancy helps **only** when the kernel is waiting on latency; it is a means, not a goal.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100).

# Check yourself

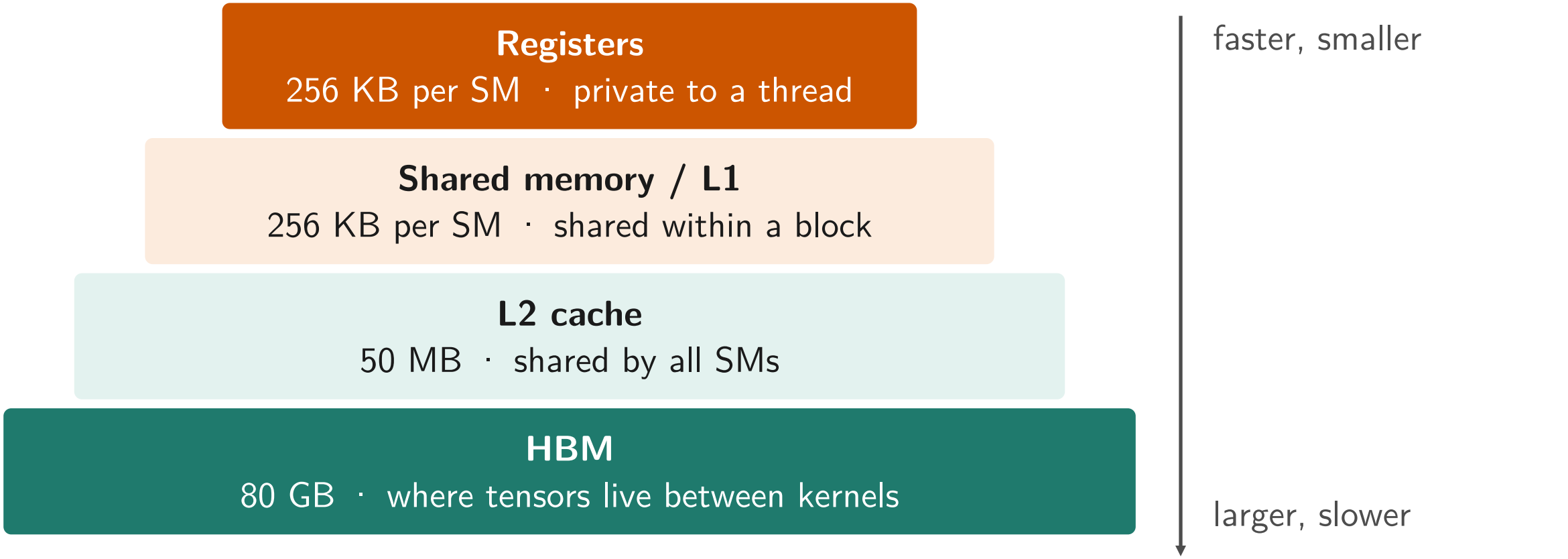
- How many threads form a warp? A block runs on how many SMs?
- `if i % 2 == 0: A() else: B()` — what does it cost, and how would you restructure it?
- Bandwidth 2 TB/s, latency 500 ns: how many bytes must be in flight?

Part 3

# The memory hierarchy

Where does the data live, and how fast can we reach it?

# The memory hierarchy




**Key idea**

Load from HBM **once**, reuse many times from registers or shared memory.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

# The numbers to remember

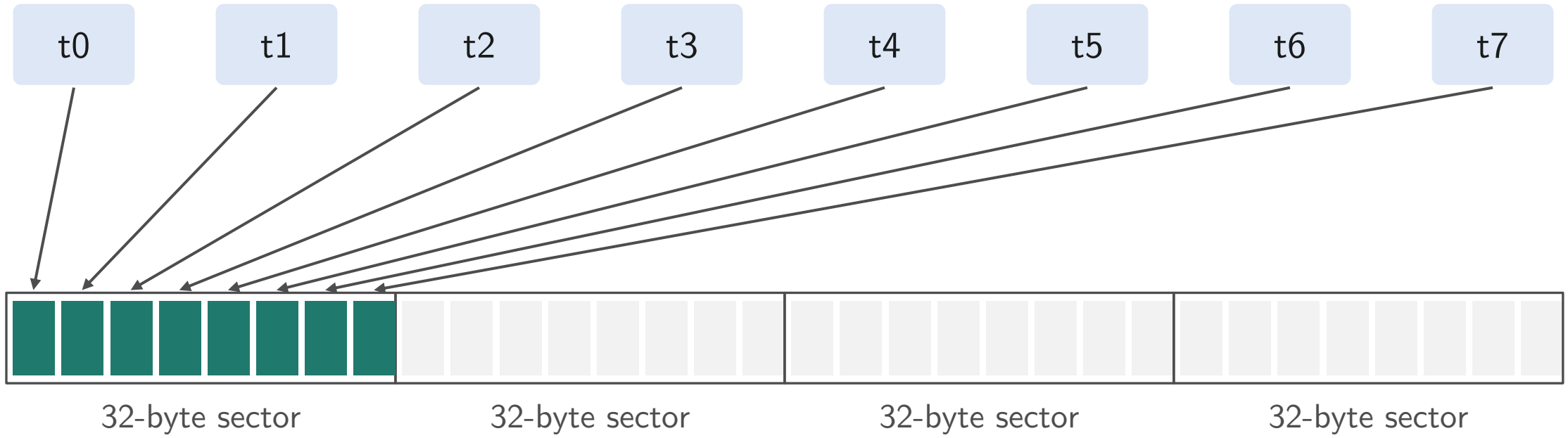
| Level | Capacity | Latency (approx.) | Bandwidth (approx.) |
|---|---|---|---|
| Registers | 256 KB / SM | $\approx$ 1 cycle | — |
| Shared memory / L1 | 256 KB / SM | $\approx$ 30 cycles | $\approx$ 33 TB/s (all SMs) |
| L2 cache | 50 MB | $\approx$ 200–300 cycles | several TB/s |
| HBM | 80 GB | $\approx$ 500–800 cycles | 3.35 TB/s |
| Host RAM over PCIe 5 | — | microseconds | $\approx$ 0.064 TB/s |

**Key idea**

Host $\leftrightarrow$ GPU copies are $\approx$ **50×** slower than HBM: keep training data on the device.

Source: H100 whitepaper and datasheet; PCI-SIG PCIe 5.0; latencies are rounded, from microbenchmarks (e.g. Luo et al., IPDPS 2024).

# Coalescing: neighbours read neighbours



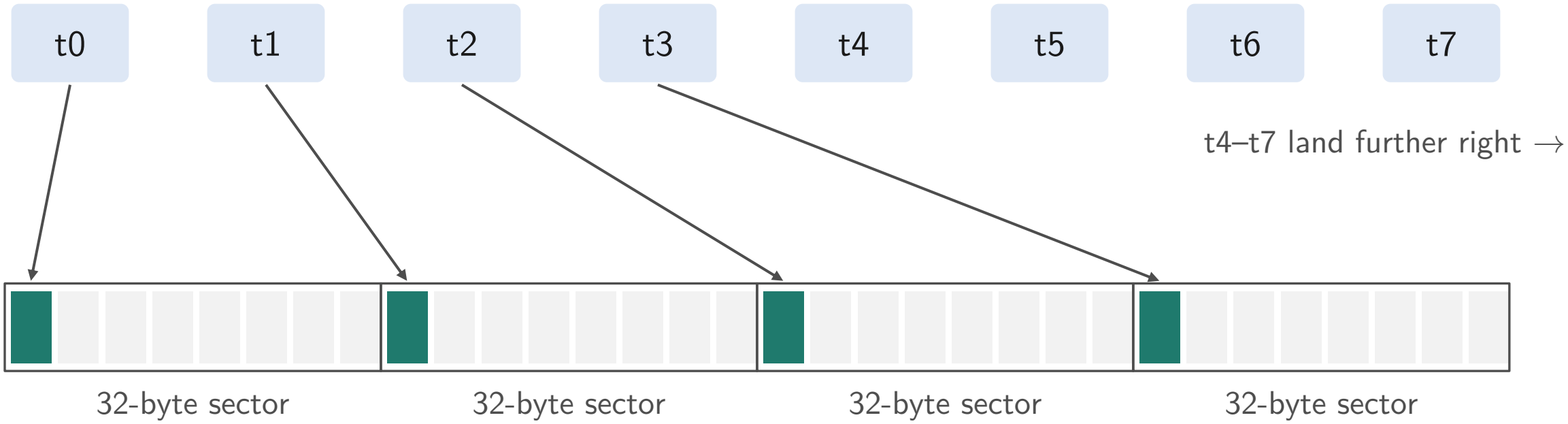

top: 8 of a warp's 32 threads · bottom: memory, one square per float

**Memory**

8 threads → one full sector; a warp of 32 → **128 bytes, all used**. Efficiency 100%.

Source: NVIDIA CUDA C++ Best Practices Guide, "Coalesced Access to Global Memory".

# Strided access wastes bandwidth




top: 8 of a warp's 32 threads · bottom: memory, one square per float

**Watch out**

Each thread pulls a whole 32-byte sector for 4 useful bytes: 1,024 bytes moved, 128 used → **12.5%**.

Source: NVIDIA CUDA C++ Best Practices Guide, "Strided Accesses".

# Row-major layout and strides

| | | | | | |
|---|---|---|---|---|---|
| 0,0 | 0,1 | 0,2 | 0,3 | 0,4 | 0,5 |
| 1,0 | 1,1 | 1,2 | 1,3 | 1,4 | 1,5 |
| 2,0 | 2,1 | 2,2 | 2,3 | 2,4 | 2,5 |
| 3,0 | 3,1 | 3,2 | 3,3 | 3,4 | 3,5 |

A: $4 \times 6$, row 1 highlighted

$$\text{offset}(i, j) = i \cdot N + j$$

$$\text{strides} = (N, 1)$$

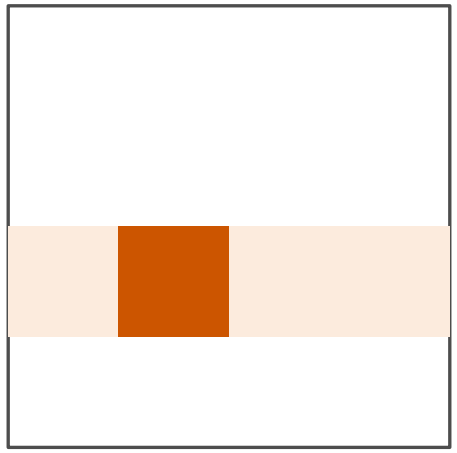


the same 24 numbers in memory: row 1 is contiguous

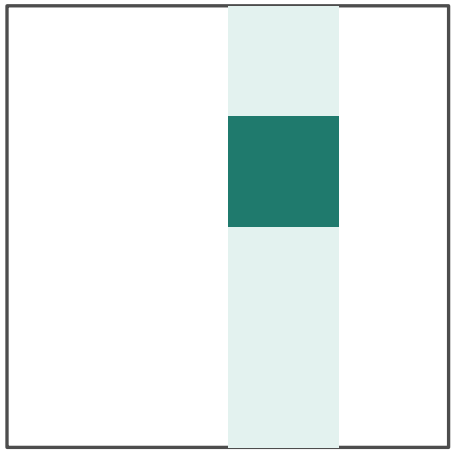
**Key idea**

A **row** has stride 1 (coalesced); a **column** has stride N (strided). `A.T` swaps strides, not data.

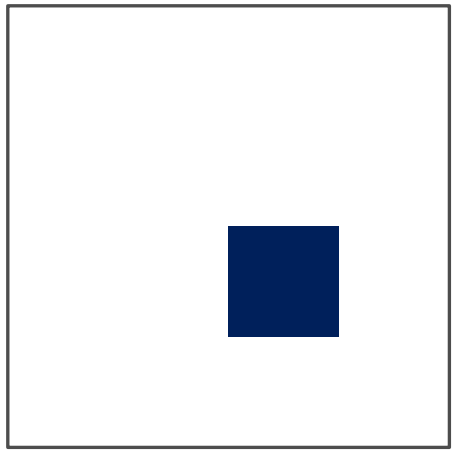
# Tiling: reuse data on chip



**A (M × K)**



**B (K × N)**



**C (M × N)**

**Naive**: each output re-reads a whole row of A and column of B from HBM.

**Tiled**: load one T × T tile of A and of B into shared memory; use each value **T times**.

**Key idea**

HBM traffic falls by a factor of ≈ **T**: tiles move once, then are reused on chip.

# Tiling, quantified

$$\text{naive:}\quad \text{reads} \approx 2\text{MNK} \;\Rightarrow\; I = 2\text{MNK} \,/\, 8\text{MNK} = 1/4$$

$$\text{tiled:}\quad \text{reads} \approx 2\text{MNK} \,/\, T \;\Rightarrow\; I \approx T \,/\, 4$$

$$T = 16 \to 4, \quad T = 32 \to 8, \quad T = 64 \to 16 \quad (\text{FLOP/byte})$$

**Watch out**

Bigger tiles mean more reuse, but more shared memory and registers → **lower occupancy**. You will tune T in T1.

# Tensor cores and precision

| Format | Bits | H100 peak (dense) | Typical use |
|---|---|---|---|
| FP32 (ordinary lanes) | 32 | $\approx$ 67 TFLOP/s | Reference math |
| TF32 (tensor cores) | 19 | $\approx$ 495 TFLOP/s | Faster FP32 matmuls |
| BF16 / FP16 (tensor cores) | 16 | $\approx$ 989 TFLOP/s | Training default |
| FP8 (tensor cores) | 8 | $\approx$ 1,979 TFLOP/s | Large-model training |

**Watch out**

These peaks apply **only to matrix products**. Elementwise work (tanh, softmax, adds) runs on ordinary lanes and is usually memory-bound.

Source: H100 datasheet (SXM5); tensor-core values are the datasheet's sparse peaks divided by 2.

# Check yourself

- Who manages shared memory? Who manages L2?
- 32 threads each read one float from a different **column** of a row-major matrix. What fraction of the bytes moved is used?
- What intensity does T = 32 tiling give in FP32? Why not simply use T = 128?

Part 4

# The roofline model

Compute-bound or memory-bound? Predict before you measure.

# Arithmetic intensity

$$I = F \,/\, Q$$

$$F = \text{FLOPs performed}, \quad Q = \text{bytes moved to/from HBM}$$

| Kernel | F | Q (minimum) | I (FLOP/B) |
|---|---|---|---|
| Vector add z = x + y (FP32) | n | 12n | $1/12 \approx 0.08$ |
| Row softmax, B × V (FP32) | $\approx 5BV$ | 8BV | $\approx 0.6$ |
| Matmul n × n (BF16, ideal reuse) | $2n^3$ | $6n^2$ | n / 3 |

**Key idea**

Q is the **minimum** traffic: every input read once, every output written once. Real kernels can only do worse.

# Two lower bounds on runtime

$$T \geq F \,/\, \pi \qquad \text{(arithmetic)}$$

$$T \geq Q \,/\, \beta \qquad \text{(memory)}$$

$$T \geq \max\left( F \,/\, \pi \;,\; Q \,/\, \beta \right)$$

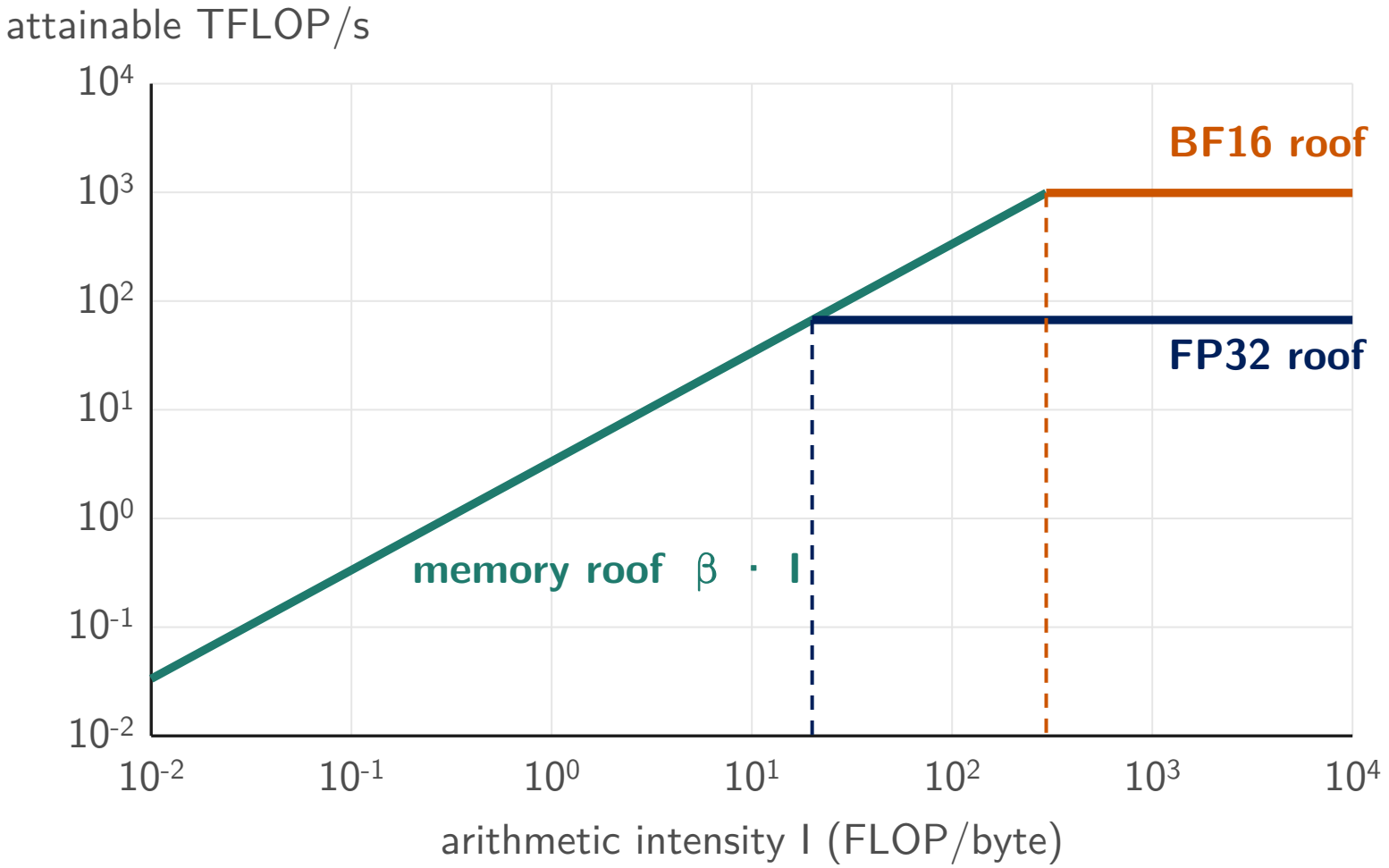
$$\text{attainable FLOP/s} = F \,/\, T \leq \min\left( \pi \;,\; \beta \cdot I \right)$$

**Key idea**

**Ridge point** $I^* = \pi \,/\, \beta$: below it, memory limits you; above it, arithmetic does. Assumes perfect overlap — an optimistic bound.

Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

# The roofline for an H100




FP32 roof: 67 TFLOP/s

BF16 roof: 989 TFLOP/s

ridges $\approx$ 20 and 295

**$\leftarrow$ memory-bound: move fewer bytes**

**compute-bound $\rightarrow$ use faster arithmetic**

Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

# Worked example 1: vector add

- $n = 10^8$ floats (FP32); $F = 10^8$ FLOPs
- $Q = 3 \times 4 \times 10^8 = 1.2 \times 10^9$ bytes
- $I = F / Q = 1/12 \approx 0.08$ FLOP/B $(\ll 20)$
- $F / \pi = 10^8 / 67 \times 10^{12} \approx 1.5$ µs
- $Q / \beta = 1.2 \times 10^9 / 3.35 \times 10^{12} \approx$ **0.36 ms**

**Verdict**

**Memory-bound** by $\approx 240\times$. A measured time near 0.36 ms means the kernel is already **optimal**: it runs at the memory roof.

# Worked example 2: matrix multiply

$$F = 2n^3, \quad Q = 6n^2, \quad I = n \,/\, 3$$

$$\text{compute-bound when } n \,/\, 3 > 295 \;\Leftrightarrow\; n \gtrsim 886$$

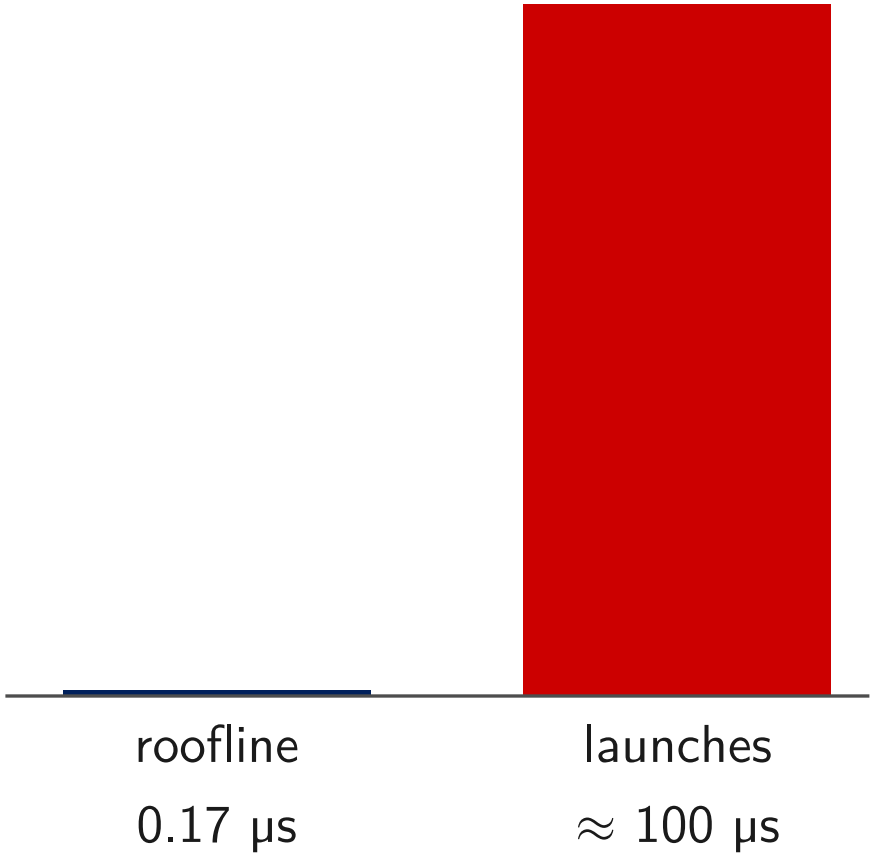
| n | I (FLOP/B) | Bound | Predicted time |
|---|---|---|---|
| 256 | 85 | memory (on paper) | 0.12 µs → launch overhead dominates |
| 1024 | 341 | compute (just) | ≈ 2.2 µs |
| 4096 | 1365 | compute | ≈ 0.14 ms |

**Watch out**

Tiny problems hit a third limit: **fixed overhead** — a kernel launch costs several µs.

# Worked example 3: our training step

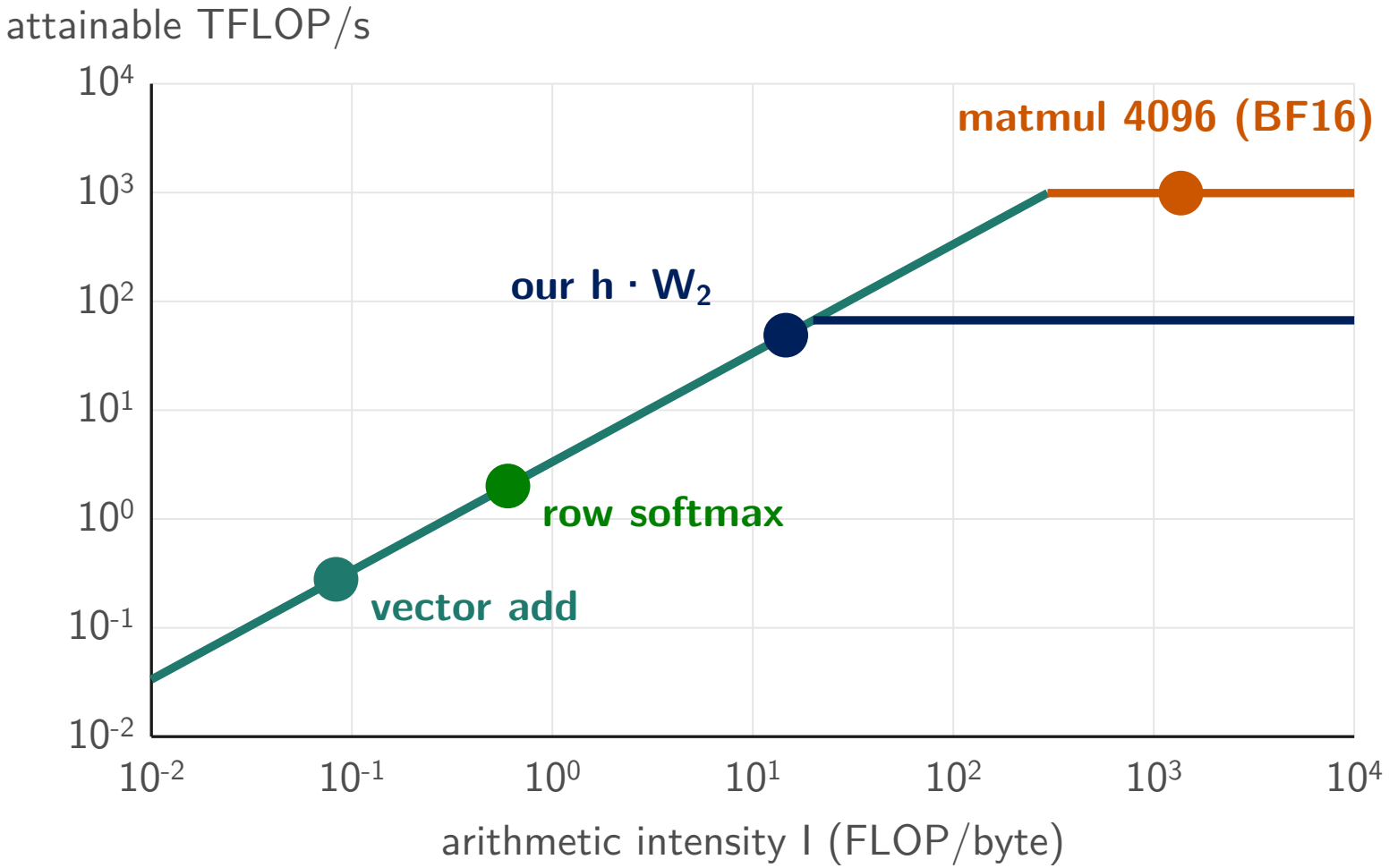
- Logits $h \cdot W_2$: $(64 \times 64) \cdot (64 \times 322)$, FP32
- $F \approx 2.64$ M FLOPs; $Q \approx 181$ KB; $I \approx 14.6$
- Roofline time $\approx 0.05$ µs; whole step $\approx$ **0.17 µs**
- $\approx 25$ kernel launches $\times \approx 4$ µs $\approx$ **100 µs**




**In our pipeline**

**Overhead-bound**: launches cost $\approx 600\times$ the roofline time. Fix: **fuse**, bigger batches, **compile**.

# Placing kernels on the roofline




- **Tensor cores** help only the large matmul.
- **Fusion** helps the left side: it cuts Q.

Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

# Fusion: fewer bytes, same FLOPs

**Unfused: 3 kernels**

$t_1 = a \cdot x$ read x, write $t_1$

$t_2 = t_1 + b$ read $t_1$, write $t_2$

$y = \tanh(t_2)$ read $t_2$, write y

**Q = 24n bytes**

**Fused: 1 kernel**

$y = \tanh(a \cdot x + b)$

read x once, write y once

$t_1$, $t_2$ stay in registers

**Q = 8n bytes**

$$\text{same } F; \quad Q\text{: } 24n \to 8n; \quad \text{memory-bound} \Rightarrow \text{predicted speedup} \approx 3\times$$

**In our pipeline**

T1: write one Triton **reduction or fused elementwise** kernel, and predict its speedup **before** timing it.

# What the roofline leaves out

| Effect | Symptom | Covered in |
|---|---|---|
| Launch / Python overhead | Small problems far below both roofs | Today; Lecture 2 |
| Uncoalesced access | Q far above the minimum | Today; Lab 3 |
| Low occupancy | Latency not hidden; below the roof | Today; T1 |
| Branch divergence | Idle lanes; below the compute roof | Today |
| Host $\leftrightarrow$ device copies | GPU idle between kernels | Lecture 2 |

**Key idea**

When measurement and prediction disagree, the **gap is the diagnosis**.

IN-CLASS BUILD

# Predict a kernel bottleneck

Work in pairs. Predict first; measure in Lab 3.

# Exercise: predict three kernels

| Kernel (FP32) | F | Q | I | Bound? | Time |
|---|---|---|---|---|---|
| 1. ReLU $y = \max(x, 0)$, $n = 10^8$ | | | | | |
| 2. Row sums of a $4096 \times 4096$ matrix | | | | | |
| 3. Matmul $2048 \times 2048$, tiles $T = 64$ | | | | | |

**USE**

$\pi = 67$ TFLOP/s, $\beta = 3.35$ TB/s

One comparison = one FLOP.

**STRETCH**

For kernel 3, compare the tiled estimate with the ideal-reuse estimate.

# Exercise: solutions

| Kernel (FP32) | F | Q | I | Bound | Time |
|---|---|---|---|---|---|
| 1. ReLU, $n = 10^8$ | $10^8$ | $8 \times 10^8$ B | 0.125 | memory | $\approx$ 0.24 ms |
| 2. Row sums, $4096^2$ | $1.7 \times 10^7$ | $6.7 \times 10^7$ B | 0.25 | memory | $\approx$ 20 µs |
| 3a. Matmul, $T = 64$ | $1.7 \times 10^{10}$ | F / 16 | 16 | memory | $\approx$ 0.32 ms |
| 3b. Matmul, ideal reuse | $1.7 \times 10^{10}$ | $5.0 \times 10^7$ B | 341 | compute | $\approx$ 0.26 ms |

**Key idea**

3a and 3b **bracket** reality. In Lab 3 you will measure kernel 1 and see how close you get to 0.24 ms.

# Measuring fairly

```
def bench(fn, repeats=20):
    fn(); torch.cuda.synchronize()   # warm up
    times = []
    for _ in range(repeats):
        t0 = time.perf_counter()
        fn()
        torch.cuda.synchronize()
        times.append(time.perf_counter() - t0)
    return statistics.median(times)

x = torch.randn(10**8, device="cuda")
t = bench(lambda: torch.relu(x))
print(8 * x.numel() / t / 1e9, "GB/s")
```

- **Synchronize**: launches are asynchronous.
- **Warm up**: the first call sets up.
- **Repeat**: report median and spread.
- **Convert** to GB/s and compare with the roof.

**In our pipeline**

T1 checkpoint: a correct kernel **and** a correct timing harness.

# Key takeaways

- GPUs trade single-thread **latency** for **throughput**; training is a throughput problem.
- Threads run in **warps of 32** (SIMT); divergence within a warp wastes lanes.
- GPUs hide memory latency by **switching among many warps** (Little's law, occupancy).
- **Coalescing** and **tiling** reduce the bytes that reach HBM.
- Roofline: $\mathbf{T \geq \max(F/\pi,\ Q/\beta)}$ — compute $I = F/Q$, compare with $\pi/\beta$, predict before measuring.
- Our small MLP is **overhead-bound** — the motivation for fusion and compilation.

# Next time, and readings

| When | What |
|---|---|
| Lecture 2 (this week) | GPU programming model: blocks, grids, masks, reductions in Triton |
| Friday (this week) | T3 due |
| After Lecture 2 | T1 and Lab 3 released |
| Week 13 | The memory hierarchy; arithmetic intensity and the roofline |

**READ BEFORE LECTURE 2**

CUDA C++ Programming Guide: *Programming Model* and *Hardware Implementation*.
Williams, Waterman & Patterson, "Roofline," CACM 2009.